

OptSQL: A Meta-Cognitive Multi-Agent System for Efficiency-Oriented Text-to-SQL

ANONYMOUS AUTHOR(S)

Text-to-SQL aims to translate natural language questions into executable SQL queries. Recent LLM-based systems have improved robustness through workflow-based generation, but they still rely on largely fixed pipelines, use feedback mainly for post-hoc repair, and focus primarily on correctness while overlooking query efficiency. To address these limitations, we propose OptSQL, a meta-cognitive multi-agent Text-to-SQL system that dynamically coordinates evidence construction, SQL generation, process-level reflection, and plan-aware optimization. A Meta-Cognitive Controller continuously observes the task state and database feedback to adapt workflow strategies, reflect on intermediate reasoning states, and perform execution- and plan-aware rewriting. We evaluate OptSQL on BIRD and EESQLBENCH. OptSQL achieves the best EX of 73.29% and AR@1 of 67.96% on BIRD, and the best EX of 67.89% and AR@1 of 50.59% on EESQLBENCH. These results show that meta-cognitive control improves the correctness and efficiency of Text-to-SQL systems.

1 Introduction

Text-to-SQL aims to translate natural language (NL) questions into executable SQL queries, enabling non-expert users to retrieve and analyze relational data without manually writing database programs [1, 10, 30]. Recent LLM-based systems have achieved substantial progress through in-context learning, task decomposition and specialized agents, moving the field beyond single-shot generation toward workflow-based SQL construction [8, 32, 40, 44].

From Single-shot Generation to Agentic Workflows. As shown in Figure 1, modern Text-to-SQL systems typically organize SQL construction into three stages: pre-generation, generation, and post-generation. In the pre-generation stage, the system grounds the natural language query in the database context through schema linking, value grounding, and information retrieval, thereby reducing the search space and enriching the input evidence [2, 14, 29, 40]. In the generation stage, the model applies prompting strategies, query planning, and LLM-based reasoning to produce one or more candidate SQL queries [3, 8, 9, 32, 44, 49]. In the post-generation stage, these candidates are further checked and refined through execution validation, repair, and consistency- or ensemble-based selection [3, 9, 32, 40, 44, 45, 49, 50]. Compared with single-pass generation, such workflows are more robust because they decompose SQL construction into modular steps, strengthen database grounding, diversify candidate generation, and provide post-hoc opportunities for validation and refinement. Despite these advances, existing Text-to-SQL systems still face three coupled limitations:

L1: Limited Adaptation in Workflows and Strategy Selection. Many existing Text-to-SQL agents often predefine both the workflow structure and the strategy choices, where modules such as schema linking, few-shot retrieval, candidate generation, validation, and revision are applied in a largely fixed manner [3, 8, 29, 32, 44]. Although such designs improve robustness over single-shot generation, they offer limited support for adapting reasoning strategies to the database environment, question difficulty, or intermediate feedback. This may misallocate reasoning effort: simple queries over intuitive schemas can incur unnecessary token and latency overhead, whereas difficult queries may require deeper grounding, candidate comparison, and additional validation or reflection. This mismatch is illustrated by the different effects of domain-specific hints in BIRD [20] dataset. For an intuitive database such as superhero, where tables and columns are relatively descriptive, adding domain hints provides little additional benefit in our experiments. In contrast, for a domain-heavy database such as thrombosis_prediction, where terms such as LDH and gender-dependent uric-acid ranges require medical knowledge, the same hints improve accuracy from 20% to nearly 50%. This contrast suggests that Text-to-SQL agents should adapt their grounding and generation strategies to the database environment, rather than applying the same strategy uniformly.

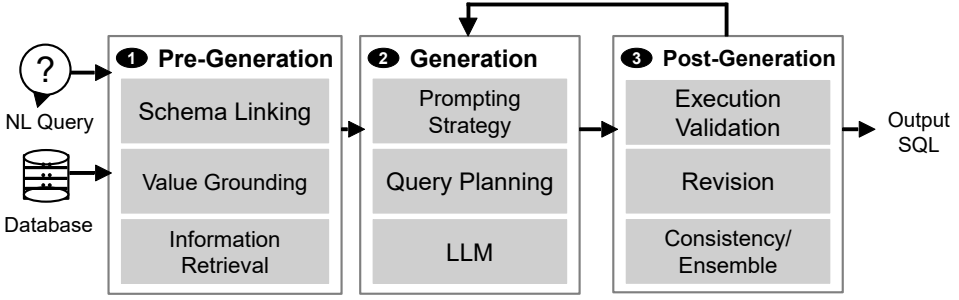


Fig. 1. Overview of a typical Text-to-SQL pipeline, illustrating the Pre-Generation, Generation, and Post-Generation stages

L2: Post-hoc rather than Process-level Reflection. Feedback in many Text-to-SQL systems is mainly used after a candidate SQL has been generated, such as repairing execution errors, revising invalid queries, or selecting among completed candidates [12, 32, 40, 44]. Such feedback improves final-output validation, but provides limited support for monitoring and regulating the generation process itself. In particular, the agent often lacks a controller that continuously tracks whether schema linking is reliable, whether candidate disagreement is increasing, whether repeated repairs are converging, or whether the current strategy should be changed. As a result, early mistakes in schema linking, value grounding, or query planning may propagate into later steps and become cascading hallucinations. Moreover, without global process supervision, local repair actions may fix surface-level execution errors while leaving the overall reasoning trajectory insufficiently grounded in the user question and database context.

L3: Limited Awareness of Query Efficiency¹. Most existing Text-to-SQL systems are primarily optimized for correctness: they aim to generate SQL queries whose execution results match the ground-truth answers [17–19, 35, 37]. However, in practical database applications, a semantically correct SQL query is not necessarily efficient. Equivalent SQL formulations may lead to very different execution costs due to variations in join orders, predicate placement, or aggregation strategies [4, 13, 47]. Yet, current Text-to-SQL systems mainly focus on executability and result consistency, rarely treating query plans, estimated costs, or bottleneck patterns as signals for rewriting generated SQL. As a result, the final query may be correct but unnecessarily expensive, especially in enterprise settings with large schemas, large tables, and long multi-step analytical workflows [12, 21].

Key Insights: Meta-cognitive and Environment-aware Text-to-SQL. Inspired by enactive cognition and meta-cognition, we view Text-to-SQL not as a static translation from question and schema to SQL, but as an adaptive interaction process between the agent and the database environment [7, 43]. Enactive cognition suggests that understanding is shaped through interaction with the environment, while meta-cognition emphasizes the ability to monitor and regulate one’s own reasoning process. In the context of Text-to-SQL, this means that an agent should perceive database-specific signals, act by generating and rewriting SQL, observe feedback from execution and query plans, and adjust its strategy when uncertainty, failure, or inefficiency is detected. These perspectives lead to three design objectives. First, to address L1, the agent should perform *environment-aware dynamic workflow adaptation*: it should select lightweight or deep workflows according to the perceived database environment, question complexity, and intermediate feedback, rather than applying the same strategy uniformly. Second, to address L2, the agent should support *process-level self-reflection and dynamic control*: it should monitor schema-linking

¹Unless otherwise specified, efficiency in this paper refers to the execution efficiency of the generated SQL query, rather than the efficiency of LLM-based query generation.

reliability, candidate disagreement, repair convergence, and strategy suitability during generation, so that early mistakes can be corrected before they propagate. Third, to address L3, the agent should enable *execution- and plan-aware SQL optimization*: it should use query plans, cost signals, semantic validation, and reusable rewrite knowledge to improve query efficiency while preserving correctness.

Approach. To realize these objectives, we propose OPTSQL, a meta-cognitive multi-agent system for efficiency-oriented Text-to-SQL. At the core of OPTSQL is a Meta-Cognitive Controller that maintains a task-level state in working memory, observes intermediate feedback, and dynamically decides which agent actions should be invoked next. Under its control, the Generation Phase constructs executable and semantically plausible initial SQL candidates through evidence-guided schema filtering and multi-strategy SQL generation. When further efficiency improvement is needed, the Optimization Loop analyzes query plans, identifies bottleneck patterns, rewrites inefficient SQL formulations, and validates both semantic consistency and cost reduction. Finally, a Memory Engine and Tool Infrastructure support database grounding, execution analysis, retrieval of reusable optimization experience, and updates to the long-term Auto-Evaluation Knowledge Base (AEKB) after successful rewrites.

Evaluation. We evaluate OPTSQL on two complementary benchmarks, BIRD [20] and EESQLBENCH [21], covering both general cross-domain Text-to-SQL capability and efficiency-oriented SQL generation. We compare OPTSQL with fine-tuning-based methods, prompting-based methods, and agentic Text-to-SQL workflows under multiple backbone LLMs. The results show that OPTSQL achieves the best or highly competitive performance across correctness- and efficiency-oriented metrics, including EX, VES, CR, AR@0.8, and AR@1 (see Section 4). In particular, OPTSQL achieves the best EX of 73.29% and AR@1 of 67.96% on BIRD, and the best EX of 67.89% and AR@1 of 50.59% on EESQLBENCH. These results show that OPTSQL effectively bridges semantic correctness and efficiency, validating the benefits of meta-cognitive control for practical Text-to-SQL systems.

In summary, this paper makes the following contributions:

- We identify three key limitations of existing Text-to-SQL systems: limited workflow adaptation, post-hoc reflection, and insufficient awareness of query efficiency.
- We propose OPTSQL, a meta-cognitive multi-agent system that dynamically coordinates evidence construction, SQL generation, process-level reflection, and plan-aware optimization.
- We evaluate OPTSQL on BIRD and EESQLBENCH, showing consistent improvements over prompting-based, fine-tuning-based, and agentic baselines.

2 Motivation

2.1 Task Formulation

Traditional Text-to-SQL aims to translate an NL question x into an executable SQL query q over a database environment $\mathcal{D}$. Given a ground-truth query q^* , the standard objective is to generate a query whose execution result matches that of q^* :

$$f_{\theta}(x, \mathcal{D}) \rightarrow q, \quad \text{s.t.} \quad \text{Exec}(q, \mathcal{D}) = \text{Exec}(q^*, \mathcal{D}),$$

where f_{θ} denotes a Text-to-SQL system parameterized by θ , $\mathcal{D}$ denotes the database environment including schema, data, indexes, and statistics, and $\text{Exec}(q, \mathcal{D})$ denotes the result of executing q on $\mathcal{D}$.

However, this formulation focuses mainly on semantic correctness and does not explicitly consider query efficiency. In practice, multiple semantically equivalent SQL queries may produce the same result but incur substantially different execution costs due to join order, predicate placement, aggregation strategy, or index utilization. We therefore formulate **Efficiency-Oriented Text-to-SQL**, where the goal is to generate a query that is both semantically correct and efficient. Given an

NL question x and a database environment $\mathcal{D}$, the goal is to generate a SQL query q_{opt} satisfying:

$$q_{\text{opt}} = \arg \min_{q \in Q_{\text{corr}}(x, \mathcal{D})} \text{Cost}(q, \mathcal{D}),$$

$$Q_{\text{corr}}(x, \mathcal{D}) = \{q \in Q(x, \mathcal{D}) \mid \text{Exec}(q, \mathcal{D}) = \text{Exec}(q^*, \mathcal{D})\},$$

where $Q_{\text{corr}}(x, \mathcal{D})$ denotes the subset of candidates that are semantically correct with respect to q^* . $\text{Cost}(q, \mathcal{D})$ represents the estimated or actual execution cost of query q on $\mathcal{D}$. Intuitively, the model should not only ensure that q_{opt} yields correct results but also that it achieves minimal execution time or resource consumption under the same semantic constraints.

The minimization above specifies the ideal task objective. The reference query q^* supplies the expected result for correctness evaluation and is not assumed to be a minimizer of Cost. Our research question is whether and how adaptive agent orchestration guided by execution and query-plan feedback can improve the efficiency of generated SQL while preserving correctness. Operationally, OptSQL explores and validates a finite set of candidates, and its effectiveness is assessed through empirical improvements over baseline systems and comparisons with reference queries, without claiming exhaustive search or global optimality.

2.2 Design Objective

Existing Text-to-SQL systems are typically static pipelines that decompose SQL construction into pre-defined stages, including pre-generation, generation, and post-generation validation. While such designs improve robustness compared to single-pass generation, they lack the ability to dynamically adapt their reasoning process to different database environments, query difficulties, and intermediate execution feedback.

Beyond Static Pipelines: A Meta-Cognitive Multi-Agent Perspective. Inspired by enactive cognition and meta-cognitive theory, we reformulate Text-to-SQL as a dynamic interaction process between an intelligent system and the database environment [7, 43]. Enactive cognition emphasizes that intelligence emerges through continuous interaction with the environment, while meta-cognition enables the system to monitor, regulate, and adapt its own reasoning process. Based on these principles, we propose a Meta-Cognitive Multi-Agent Text-to-SQL System, where a central Meta-Cognitive Controller dynamically orchestrates a set of specialized agents (actions) under different execution contexts. Instead of a fixed pipeline, all stages in Text-to-SQL are decomposed into reusable and composable actions, which are invoked online based on the current state of the problem and environment feedback. This design establishes a unified closed-loop system in which a Meta-Cognitive Controller coordinates planning, execution, evaluation, and optimization over SQL programs based on continuous database feedback.

Key Design Principles. (1) Environment-aware Dynamic Workflow Adaptation. The Meta-Cognitive Controller continuously observes the evolving task state, such as database environment, question complexity, candidate behaviors and dynamically decides which actions should be invoked next. For example, for simple queries over well-structured schemas (e.g., single-table filtering), the controller may directly invoke a lightweight generation path using few-shot prompting. In contrast, for complex analytical queries (e.g., multi-join or domain-specific reasoning), the controller activates a deeper workflow that includes multi-candidate generation, iterative refinement, and optimization loops. This adaptive strategy selection avoids over- or under-utilization of reasoning resources.

(2) Process-level Self-Reflection and Dynamic Control. Unlike conventional post-hoc validation approaches, our system enables continuous monitoring of intermediate reasoning states during SQL generation. The Meta-Cognitive Controller uses the recorded task trace to diagnose whether the current reasoning trajectory is reliable, converging, and sufficiently grounded. For example, when schema or value evidence is insufficient, the controller may request additional

evidence construction through reverse schema linking, value-grounded schema linking, or relational completion. Similarly, if repeated optimization iterations fail to reduce execution cost, the controller may terminate the current loop and switch to a different reasoning configuration. This enables closed-loop control over the entire reasoning trajectory.

(3) Execution- and Plan-aware Optimization via Feedback-driven Rewriting. Beyond generating semantically correct SQL, the system treats execution efficiency as an objective. Execution results, query plans, and cost estimates are used as feedback signals to reveal whether an initially correct query may still be inefficient. When such feedback indicates potential performance bottlenecks, the system can refine the SQL through feedback-driven rewriting while preserving its intended semantics. Optimization is not treated as an isolated one-shot action. The system accumulates validated optimization experience from previous interactions with the database environment, enabling future decisions to be guided not only by static rules but also by reusable execution-aware experience. In this way, SQL optimization becomes a closed-loop process in which database feedback continuously informs rewriting, validation, and future strategy selection.

3 OptSQL

3.1 Overview

Figure 2 shows the overall architecture of OPTSQL. Given an NL question and a database environment, OPTSQL aims to generate a SQL query that is both semantically correct and execution-efficient. Instead of treating Text-to-SQL as a fixed pipeline, OPTSQL formulates it as a feedback-driven process coordinated by a Meta-Cognitive Controller. The Meta-Cognitive Controller supervises the entire workflow. It perceives the database environment, estimates the difficulty of the input question, selects suitable actions, and monitors intermediate feedback throughout both generation and optimization. Under its control, OPTSQL first constructs an initial SQL query in the Generation Phase, where an evidence-guided schema filter identifies relevant tables, columns, values, and join paths, and an initial SQL builder produces executable candidate queries. If the controller detects that the query requires further efficiency improvement, it activates the Optimization Loop. This loop consists of an Explain Analyzer Agent, a SQL Rewriter Agent, and a Validator Agent, which together analyze query plans, rewrite inefficient SQL patterns, and verify semantic consistency and cost reduction. OPTSQL is further supported by a Memory Engine and a Tool Infrastructure. The Memory Engine maintains task-level working memory and reusable optimization knowledge, including expert rules and validated historical rewrite cases. The Tool Infrastructure provides database grounding, SQL execution and analysis, and RAG-based retrieval. Through these components, OPTSQL continuously observes feedback from the database environment, reflects on its intermediate decisions, and updates its optimization experience.

The remainder of this section describes each component of OPTSQL. We first present the Generation Phase for constructing an initial SQL query (Section 3.2), followed by the Optimization Loop for execution- and plan-aware rewriting (Section 3.3). We then introduce the Meta-Cognitive Controller that orchestrates these processes through strategy selection, monitoring, and self-reflection (Section 3.4). Finally, we describe the Memory Engine and Tool Infrastructure that support retrieval, experience accumulation, and database feedback (Section 3.5). Due to space limitations, we also release a detailed case study in our GitHub repository [27] to illustrate how OPTSQL realizes its key designs.

3.2 Generation Phase

The Generation Phase aims to construct a set of executable and semantically plausible initial SQL candidates based on the question and database environment. It focuses on two core capabilities: evidence construction and multi-strategy SQL generation.

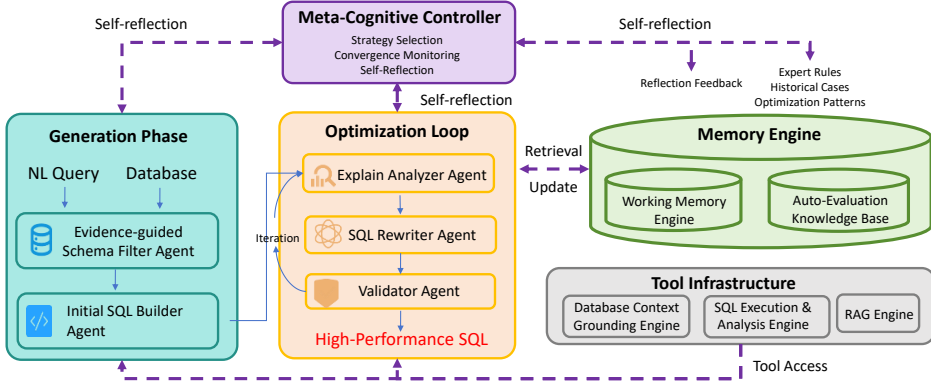


Fig. 2. Overview of OPrSQL

Evidence-guided Schema Filter Agent. This agent is responsible for grounding the input question into a compact, relevant, and executable schema context for downstream SQL generation. It combines robust schema linking with value grounding, aiming to identify not only the tables and columns mentioned or implied by the question, but also the concrete database values and conditions needed to express the user intent. To improve schema robustness, the agent supports multiple complementary schema linking strategies:

- *Direct schema linking* maps the natural language question to relevant tables and columns by reasoning over question semantics and schema descriptions. This strategy is effective when the question contains explicit lexical or semantic cues.
- *Reverse schema linking* hypothesizes a tentative SQL structure and then extracts the tables and columns implied by this structure [15]. This recovers latent schema dependencies, such as join tables, grouping columns, or aggregation-related attributes, that may be missed by direct matching.
- *Value-grounded schema linking* identifies relevant schema elements through actual database contents. Instead of relying only on table or column names, the agent checks whether values in a column match or semantically correspond to phrases in the question, which is especially useful when schema names are ambiguous or weakly informative.

Beyond schema linking, the agent further performs active value grounding to translate natural language expressions into concrete database predicates. It supports several value grounding capabilities:

- *Exact value verification* checks whether a phrase in the question directly appears in candidate columns. This is useful for grounding categorical conditions such as names, locations, statuses, or labels.
- *Semantic value matching* retrieves column values that are semantically close to question phrases, allowing the agent to handle paraphrases, abbreviations, and domain-specific expressions.
- *Distinct-value inspection* examines representative or unique values of a column to infer how natural language concepts are encoded in the database, such as whether a Boolean concept is represented by 0/1, Y/N, or textual labels.
- *Data sampling and distribution checking* inspects example rows or value distributions to understand column semantics and avoid grounding predicates on irrelevant or sparsely populated columns.
- *Null and type analysis* checks whether candidate columns contain nulls or unexpected value formats, helping the agent avoid unreliable predicates and distinguish categorical, numerical, date, and Boolean constraints.

- *Composite predicate grounding* combines multiple grounded values or columns when a natural language phrase corresponds to a compound condition, such as a grade span, date range, or multi-attribute status.

For example, a phrase such as “kindergarten to 8th grade” may be grounded as `Low_Grade = 'K'` and `High_Grade = '8'`, while “magnet program” may correspond to a binary or categorical value after inspecting the value distribution of a related column. These grounded facts are passed to the SQL generator as explicit evidence, reducing the risk of hallucinated predicates or incorrect value choices. Finally, the agent further completes the relational context required for SQL construction. The selected schema elements may be semantically relevant but structurally disconnected; for example, two relevant tables may require intermediate tables or primary–foreign key columns to form a valid join path. Therefore, the agent supplements necessary join keys and bridge tables according to the database relationships, so that the resulting schema context can support executable SQL generation. The output of this agent is an evidence-enriched schema context that integrates schema relevance, verified value grounding, ambiguous-case evidence, and join-path information for downstream SQL generation.

Initial SQL Builder Agent. Initial SQL Builder Agent is responsible for producing executable and semantically plausible initial SQL candidates. It provides a set of complementary SQL construction capabilities, rather than a fixed generation pipeline. Depending on the task complexity, schema ambiguity, and control signals from the Meta-Cognitive Controller, the agent may invoke one or more generation strategies to balance robustness and efficiency. The agent supports the following SQL generation strategies:

- *Skeleton-based generation* follows a top-down construction process. It first analyzes the required SQL components, such as selection, filtering, joining, grouping, and ordering, then sketches a SQL skeleton, and finally fills in concrete tables, columns, predicates, and functions. This strategy is useful for queries that require clear structural planning.
- *ICL-based generation* leverages retrieved demonstrations to guide SQL construction. By retrieving structurally similar examples, the agent can reuse successful query patterns and adapt them to the current grounded schema. This strategy is especially helpful when the question resembles previously observed SQL structures.
- *Decomposition-based generation* breaks a complex question into simpler sub-questions and generates SQL fragments for each sub-task before composing them into a complete query. This strategy is designed for complex analytical questions involving nested reasoning, multiple conditions, or multi-step aggregation.

After candidate generation, the agent can apply a lightweight repair loop to improve executability. It checks basic syntactic and execution errors, and revises invalid candidates using the original question, grounded schema evidence, and database error messages. This repair process removes obvious generation failures before candidate selection, while leaving global strategy switching and deeper optimization to the Meta-Cognitive Controller. When the controller activates multiple generation strategies, the agent may produce multiple candidate SQLs. In this case, the agent further performs *execution-based selection*. Candidates are grouped according to their execution results, and each group is treated as a behavioral cluster. If one cluster forms a dominant majority, the agent selects a representative query from that cluster as the initial SQL. Otherwise, representative candidates from the top clusters are compared through pairwise semantic judgment, and each candidate is ranked by a confidence-aware score. The candidate with the highest score is selected as the output of the Initial SQL Builder Agent.

Table 1. Representative optimization patterns used by the Explain Analyzer Agent.

Pattern	Rewrite Direction	Applicability Condition
Redundant projection	Replace SELECT * with necessary projected columns.	Applied when required output columns can be inferred from the question or generated SQL.
Scalar MAX/MIN subquery	Rewrite to ORDER BY ... LIMIT 1.	Applied only when tie behavior and aggregation semantics are preserved.
Correlated subquery	Rewrite to JOIN or CTE.	Applied when an equivalent rewrite pattern is retrieved or can be validated.
Missing filter push-down	Move selective predicates closer to base tables.	Applied when predicate movement does not change join or aggregation semantics.
Late aggregation	Pre-aggregate before join.	Applied when grouping keys and aggregation semantics remain equivalent.
Inefficient OR predicate	Rewrite to UNION or UNION ALL.	Applied when duplicate semantics are correctly handled.
Redundant join path	Simplify unnecessary joins or bridge tables.	Applied only when projected columns and filtering semantics are unaffected.
Function on indexed column	Rewrite to an equivalent range or comparison predicate.	Applied when the transformation preserves value semantics.
Inefficient ordering	Align ORDER BY / LIMIT with available ordering or indexes.	Applied when ordering semantics are unchanged.

3.3 Optimization Loop

The Optimization Loop aims to improve the execution efficiency of an initial SQL query while preserving its semantics. It is activated by the Meta-Cognitive Controller when the generated SQL may benefit from further optimization. The loop consists of three agents: the Explain Analyzer Agent, the SQL Rewriter Agent, and the Validator Agent.

Explain Analyzer Agent. This agent is responsible for diagnosing potential inefficiencies in the current SQL query and producing optimization suggestions for the SQL Rewriter Agent. Given an initial SQL query, it first obtains the query plan from the database and normalizes raw plan operators into stable bottleneck signals. These plan-level signals are then combined with lightweight SQL-structure analysis and schema statistics, such as table size, available indexes, and nullable columns, to decide whether the observed behavior is likely to be a real bottleneck. The agent then maps each bottleneck signal to one or more candidate rewrite directions. For instance, consider a query such as “SELECT order_id, customer_id, order_date FROM orders WHERE status = ‘shipped’ ORDER BY order_date LIMIT 10”. If the query plan contains both SCAN orders and USE TEMP B-TREE FOR ORDER BY, the agent derives two signals: full_table_scan and temp_sort. When orders is a large table, the full scan indicates a potentially expensive access path and may trigger access-path-aware suggestions, while the temporary sort may suggest aligning the ORDER BY/LIMIT pattern with available ordering or indexes. The output of the agent is therefore a set of suggestions, such as *missing filter pushdown* or *inefficient ordering*, rather than a rewritten SQL query.

As shown in Table 1, the agent maintains a set of optimization patterns. Each pattern describes a possible inefficiency, a candidate rewrite direction, and an applicability condition that constrains when the rewrite can be attempted without changing query semantics. These suggestions are not directly applied as deterministic rules. Instead, they serve as evidence for the SQL Rewriter Agent, which decides how to rewrite the query under semantic-preservation constraints.

SQL Rewriter Agent. The SQL Rewriter Agent transforms the current SQL query into a potentially more efficient variant based on the suggestions produced by the Explain Analyzer Agent. Its input includes the original SQL, the detected bottleneck signals, the corresponding rewrite directions, the applicability conditions, and optionally retrieved rewrite cases from the Memory Engine. The rewriter does not freely optimize the query; instead, it is constrained to apply rewrite actions that are supported by the available evidence.

Validator Agent. The Validator Agent determines whether a rewritten SQL query should be accepted. It performs two checks. First, it checks semantic consistency between the original SQL and the rewritten SQL, ensuring that the rewrite does not change the intended query result. Second, it evaluates whether the rewritten SQL improves execution efficiency using available database

feedback including estimated optimizer cost and execution latency. If the rewritten SQL fails the semantic-consistency check, it is rejected and returned to the Meta-Cognitive Controller for reflection. The controller may then ask the SQL Rewriter Agent to retry with the original SQL, the failed rewrite, and the validation feedback. If the rewritten SQL preserves semantics but does not reduce cost, the controller may retry with another rewrite direction, retrieve additional cases from the Memory Engine, or terminate the optimization loop when further improvement is unlikely. Only rewrites that pass both semantic consistency and cost-improvement checks are accepted as optimized outputs.

3.4 Meta-Cognitive Controller

The Meta-Cognitive Controller is the central coordination module of OPTSQL. It maintains a global task state, selects which agent capabilities should be invoked, monitors intermediate feedback, and decides whether the system should continue, switch strategy, or terminate. Algorithm 1 formalizes the Meta-Cognitive Controller as a state-based control loop shared by generation and optimization. The workflow defines the available agent capabilities and their execution rules; within this workflow, the LLM interprets feedback and chooses what to do next. We follow the algorithm below from state initialization through action selection to validation and termination.

Initialize the State (line 1). For a question x and database $\mathcal{D}$, the Memory Engine initializes the task's working memory, whose structured state is

$$s = \langle E, Q, q, P, V, H \rangle.$$

The controller reads and updates this same state. All symbols refer to the current iteration and are updated in place, as in Algorithm 1. Here, E contains the question and database context, task-complexity observations, and grounding evidence, including schema/value alternatives, join paths, and evidence reliability. Q stores generated candidates with their strategies, execution and local-repair outcomes, result clusters, and confidence scores; q identifies the currently selected query. P contains the current query's plan, cost observations, bottlenecks, applicable rewrite directions, and retrieved AEKB cases. V stores validation reports linked to each source-rewrite pair, including consistency results, cost changes, and acceptance decisions. H is the action history, recording strategies, arguments, reflection records, outputs, and feedback, including unsuccessful attempts. Initially, E contains the input question and available database context, $q = \perp$, and the remaining fields are empty. Throughout the algorithm, q denotes the query component of s , so assigning q directly updates that component; AEKB K remains separate from the task-local state.

Set Up the Current Phase (lines 2–4). The controller first enters generation and then, if needed, optimization. The optimization decision (line 3) is evaluated only after generation has selected an initial SQL; the LLM uses that query, database context, and accumulated feedback to decide whether to continue. For each phase, U contains its available action identifiers: evidence gathering and SQL construction during generation, or plan analysis, case retrieval, and direction-specific rewriting during optimization. An identifier specifies a capability and strategy, with separate identifiers for an initial attempt and its feedback-guided revision; the arguments are supplied from the current state. This finite catalog encodes the capabilities described in Sections 3.2 and 3.3, while leaving their selection and order to the controller.

Observe Signals and Identify Eligible Actions (lines 5–9). At each iteration, $\sigma = \text{Signals}(s)$ summarizes four Boolean indicators (g, e, b, f): unresolved grounding or missing join paths (g), unresolved candidate execution errors or result disagreement (e), supported current-plan bottlenecks (b), and a failed or non-improving rewrite of the current source query (f) (line 6). These indicators describe the current unresolved issues, rather than every past failure in the history; supporting details remain in s . The program then forms A by filtering the remaining actions in U against their

Algorithm 1 Meta-cognitive control of generation and optional optimization

Require: Question x , database $\mathcal{D}$, AEKB K

Ensure: Selected SQL q , or $\perp$ if generation fails

```

442 // Initialize the State
443 1: Initialize working-memory state  $s$  from  $(x, \mathcal{D})$ ;  $q \leftarrow \perp$ 
444 // Set Up the Current Phase
445 2: for  $p \in [\text{generation, optimization}]$  do
446 3:   If  $p = \text{optimization}$  and  $\text{Optimize}_{\text{LLM}}(s) = \text{false}$ , return  $q$ 
447 4:    $U \leftarrow \text{Catalog}(p)$  //  $U$ : remaining action identifiers
448 // Observe Signals and Identify Eligible Actions
449 5:   while  $U \neq \emptyset$  do
450 6:      $\sigma \leftarrow \text{Signals}(s)$  //  $\sigma$ : Boolean flags for grounding, candidate, plan, and rewrite issues
451 7:      $A \leftarrow \text{Eligible}(U, s) \cup \text{AllowedStop}(p, s)$  //  $A$ : eligible actions, including a permitted stop
452 8:     if  $A = \emptyset$  then
453 9:       break
454 10:    end if
455 // Reflect, Select, and Adapt
456 11:     $r \leftarrow \text{Reflect}_{\text{LLM}}(s, \sigma)$  if  $\text{NeedsReflection}(\sigma)$ , else  $\emptyset$  //  $r$ : issue, supporting evidence, and next-step
457    constraint
458 12:     $\tilde{a} \leftarrow \text{Propose}_{\text{LLM}}(s, \sigma, r, A)$  //  $\tilde{a}$ : proposed action identifier and arguments
459 13:     $a \leftarrow \text{CheckOrFallback}(\tilde{a}, A, s, p)$  //  $a$ : checked proposal or valid fallback
460 14:    if  $a = \text{STOP}$  then
461 15:      break
462 16:    end if
463 17:     $U \leftarrow U \setminus \{\text{id}(a)\}$ ;  $o \leftarrow \text{Execute}(a, s, K)$  //  $o$ : agent report containing outputs, feedback, or errors
464 // Validate the Output and Update the State
465 18:    if  $a$  is a rewrite action and  $o.\text{sql} \neq \perp$  then
466 19:       $q' \leftarrow o.\text{sql}$ 
467 20:       $v \leftarrow \text{Validate}(q, q', \mathcal{D})$  //  $v$ : execution, consistency, and cost-check results
468 21:      if  $v.\text{success} \wedge v.\text{consistent} \wedge v.\text{costImproved}$  then
469 22:         $K \leftarrow \text{StoreValidatedCase}(K, q, q', v)$ 
470 23:         $q \leftarrow q'$ ;  $o.P \leftarrow \text{Analyze}(q, \mathcal{D})$ 
471 24:      end if
472 25:       $o.v \leftarrow v$ ; attach the acceptance decision to  $o$ 
473 26:    end if
474 27:     $s \leftarrow \text{Update}(s, a, \sigma, r, o)$ 
475 28:  end while
476 // Complete the Phase and Return the Query
477 29:  if  $p = \text{generation}$  then
478 30:     $q \leftarrow \text{BuilderSelect}(s.Q)$ 
479 31:    if  $q = \perp$  then
480 32:      return  $\perp$ 
481 33:    end if
482 34:  end if
483 35: end for
484 36: return  $q$ 

```

preconditions (line 7). SQL construction requires schema context; rewriting requires an executable selected query and an applicable direction; a revision requires feedback from its initial attempt, referring to the still-current source SQL for rewrite retries. The program also permits stopping

generation once an executable candidate exists, and permits stopping optimization while retaining the current query. If A is empty, the phase ends.

Reflect, Select, and Adapt (lines 11–17). The program explicitly sets $\text{NeedsReflection}(\sigma) = g \vee e \vee f$; when true, the LLM produces the following reflection record (line 11):

$$r = \langle \text{issue, supporting feedback, next-step constraint} \rangle;$$

otherwise, r is empty; a malformed reflection record is also treated as empty. For example, a failed consistency check identifies which query behavior changed and supplies a semantic constraint for the next rewrite. Given (s, σ, r, A) , the LLM proposes an action identifier and its arguments (line 12); it can gather more evidence, switch SQL construction strategies, retrieve cases, retry a rewrite with feedback, choose another direction, or stop, subject to the current phase. Thus, σ controls whether reflection occurs and directly informs action selection even when r is empty. This state-dependent choice implements adaptation: the next capability and its inputs depend on the current evidence and reflection, rather than a fixed sequence of agent calls. The program checks the proposal against A and available state objects; an invalid proposal falls back to the first eligible non-stop action in catalog order with state-bound arguments, or to an allowed stop if none remains (line 13). For a non-stop action, the program consumes its identifier and invokes the selected agent (line 17); SQL-building actions include the Builder's local execution-error repair, while process-level strategy switching remains with the controller. Execute returns a structured report o rather than directly replacing q ; every report has $o.\text{sql} = \perp$ unless it contains a proposed rewrite, and failures are returned as error reports for the common state-update step.

Validate the Output and Update the State (lines 19–27). A proposed rewrite is explicitly extracted as $q' = o.\text{sql}$ (line 19) before validation against the current query. The Validator returns Boolean fields $v.\text{success}$, $v.\text{consistent}$, and $v.\text{costImproved}$: successful execution, matching results on the available database state, and $C(q') < C(q)$ for $C(q) = \text{Cost}(q, \mathcal{D})$ under the same measure and environment (lines 20–21). Each field is false unless its check succeeds; missing or incomparable costs make $v.\text{costImproved}$ false, so a failed check cannot accept a rewrite. On acceptance, the Memory Engine stores the validated case in AEKB, q' replaces q , and plan feedback is refreshed (lines 22–23); on rejection, q remains unchanged and the failure stays in working memory (line 25). After every action, Update merges the agent report into the appropriate fields of s and appends (a, σ, r, o, q) to H , completing the cycle back to observation (line 27). This merge preserves the selected-query field: q is assigned only by candidate selection or an accepted rewrite, so a failed action cannot overwrite it. Evidence-gathering outputs update E , generation outputs update Q , plan analysis and retrieval update P , and rewrite checks update V ; the reflection record r is retained in H . When q changes, plan information is refreshed and retrieved-case applicability is rechecked for the new query; earlier observations remain accessible through H . If plan analysis fails, the old plan is invalidated and the error is recorded; only newly supported directions are eligible for rewriting.

Complete the Phase and Return the Query (lines 30–36). When generation ends, the SQL Builder selects an executable initial query, or returns $\perp$ if none exists. The controller optionally optimizes this query and returns the latest accepted result. Each phase ends on a permitted stop or when no eligible action remains; executed action identifiers are removed from the finite set U . The LLM supplies reflection and adaptive selection, while workflow and validation rules are manually specified; AEKB updates store validated cases without changing model parameters.

3.5 Memory Engine and Tool Infrastructure

Memory Engine. The Memory Engine manages two stores: task-local working memory and the reusable AEKB K . Working memory holds exactly the controller state $s = \langle E, Q, q, P, V, H \rangle$ defined in Section 3.4. Its fields provide the controller's current view of task context and grounding (E),

generated candidates (Q), the selected query (q), plan and retrieval context (P), validation outcomes (V), and the action trace (H). The controller supplies decisions and agent reports; the Memory Engine records them in these fields through the update step of Algorithm 1. Thus, working state and working memory refer to the same task-local information, with H preserving the history behind the current state.

The AEKB contains static expert rules and reusable rewrite cases, rather than the full task state or action trace. Retrieval places relevant cases in P for the current query without changing the AEKB. A new case is added to K only when a rewrite executes successfully, preserves the source query's observed results, and reduces cost under the same measure and environment. Failed rewrites and their diagnoses remain in V and H and are not promoted to reusable knowledge. The accepted case is summarized into the template record below; this promotion does not copy the question's entire working memory into K . Each accepted case is summarized as a rewrite template record:

$$\langle T_{src}, T_{dst}, C, R \rangle.$$

Here, (T_{src}) denotes the source SQL template, (T_{dst}) denotes the rewritten SQL template, (C) specifies the applicability constraints, and (R) describes the optimization rationale. To retrieve relevant cases, the RAG Engine first normalizes the current SQL into a template. It parses the SQL into an AST, replaces table names with TABLE, replaces column names with coarse semantic tokens such as TEXT_COLUMN, NUMERIC_COLUMN, DATE_COLUMN, ID_COLUMN, and FK_COLUMN, replaces literals with LITERAL, and normalizes whitespace and case. The normalized template is then compared with the (T_{src}) templates in the AEKB using a hybrid similarity score that combines sequence similarity and token-set overlap. The top-ranked records are returned as candidate rewrite knowledge for the SQL Rewriter Agent. Through this mechanism, OPTSQL accumulates validated optimization experience and reuses it to guide future bottleneck analysis and rewrite selection.

Tool Infrastructure. The Tool Infrastructure provides executable interfaces between the agents and the database environment. It consists of three groups of tools:

- *Database Context Grounding Engine.* This engine supports schema and value evidence construction for the Evidence-guided Schema Filter Agent. It provides operations for schema linking, value verification, semantic value matching, distinct-value inspection, data sampling, null/type analysis, and relational completion. These tools help the system ground NL expressions into relevant tables, columns, values, and join paths.
- *SQL Execution and Analysis Engine.* This engine supports execution-aware optimization for the Optimization Loop. It includes a query plan analyzer, a SQL quality analyzer, a semantic consistency checker, and a cost evaluator. The query plan analyzer normalizes raw query plans into plan-level risk signals, while the SQL quality analyzer detects static SQL risks from the parsed AST. The semantic consistency checker compares the original and rewritten SQLs on the available database state, and the cost evaluator measures estimated optimizer cost and execution latency before and after rewriting.
- *RAG Engine.* This engine supports memory retrieval and update. It provides a SQL formatter for template normalization, a retriever for selecting relevant rewrite cases, and an updater for inserting validated optimization cases into the AEKB. These tools allow the SQL Rewriter Agent to reuse prior optimization experience and update the knowledge base after successful rewrites.

Together, these tools allow OPTSQL to ground reasoning in database evidence, analyze execution behavior, retrieve relevant optimization experience, and update memory after successful rewrites.

4 Experiment

4.1 Experimental Setup

Datasets. We evaluate OPTSQL on two complementary benchmarks: **BIRD** [20] and **EESQL-BENCH** [21]. BIRD is a widely used cross-domain Text-to-SQL benchmark, containing more than 12,751 question-SQL pairs over 95 large-scale databases and 37 professional domains. It features realistic database scenarios with complex schemas, domain-specific knowledge, and noisy data, making it suitable for evaluating the general Text-to-SQL capability of OPTSQL. To further evaluate the generality and efficiency-oriented capability of OPTSQL, we use EESQLBENCH, an efficiency-focused benchmark where each NL question is paired with an expert-optimized canonical SQL query. Its tasks include large databases, complex joins, nested subqueries, and optimization-sensitive predicates, enabling us to assess whether OPTSQL can generate SQL queries that are not only semantically correct but also close to human expert-level efficiency.

Metrics. We evaluate both semantic correctness and execution efficiency. Let q_i^* be the canonical SQL, $\hat{q}_i$ be the generated SQL, S_i be the database, and N be the number of test instances. We define:

$$\mathbf{1}_i = \begin{cases} 1, & \text{Exec}(\hat{q}_i, S_i) = \text{Exec}(q_i^*, S_i), \\ 0, & \text{otherwise.} \end{cases}$$

Execution Accuracy (EX) [20] measures the fraction of semantically correct outputs:

$$\text{EX} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}_i.$$

It evaluates whether the generated SQL produces the correct result, regardless of efficiency.

Valid Efficiency Score (VES) [20] measures runtime-based efficiency for valid outputs:

$$\text{VES} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}_i \cdot \sqrt{\frac{T(q_i^*, S_i)}{T(\hat{q}_i, S_i)}},$$

where $T(q, S)$ denotes execution time. VES rewards generated SQLs that are both correct and fast, but it may be affected by runtime noise. We report VES multiplied by 100; it can exceed 100 because correct generated queries may execute faster than the reference queries.

Cost Reachability (CR) [21] measures how close a generated SQL is to the expert-optimized SQL in terms of plan-level cost. For each semantically correct instance, we define the instance-level cost ratio:

$$\text{CR}_i = \frac{\text{Cost}(q_i^*, S_i)}{\text{Cost}(\hat{q}_i, S_i)}.$$

The overall CR is computed as:

$$\text{CR} = \frac{1}{\sum_{i=1}^N \mathbf{1}_i} \sum_{i=1}^N \mathbf{1}_i \cdot \sqrt{\text{CR}_i}.$$

On EESQLBENCH, $\text{Cost}(\cdot)$ is measured by Total Scanned Rows, i.e., the total number of rows processed by physical operators in the query plan. A higher CR indicates that the generated SQL is closer to expert-level efficiency.

Acceptable Reachability at k (AR@ k) [21] measures the fraction of all test instances where the generated SQL is both correct and sufficiently efficient:

$$\text{AR@}k = \frac{1}{N} \sum_{i=1}^N \mathbf{1}_i \cdot \mathbf{1}(\text{CR}_i \geq k).$$

We report AR@1 and AR@0.8. AR@1 counts correct queries whose cost is no higher than the expert SQL, while AR@0.8 allows the generated SQL to be at most 1.25× the expert cost.

Following the evaluation design of EESQLBENCH [21], we assess query efficiency relative to expert-optimized canonical SQL using CR and AR. The expert-optimized canonical SQL serves as an efficiency reference, not a guaranteed global optimum or a lower bound on execution cost.

Baselines. Our baseline selection prioritizes state-of-the-art methods that demonstrated strong performance on the BIRD leaderboard and had publicly available implementations or model checkpoints at the time of our experiments to support reproducible evaluation. We compare OPTSQL with three categories of representative Text-to-SQL methods:

- **Fine-tuning-based methods.** This category includes models that rely on in-domain training data or task-specific fine-tuning to improve Text-to-SQL performance. We include **XiYAN-SQL** [23] and **OMNI-SQL** [16], using their released checkpoints XiYANSQL-QWENCODER-32B-2504 and OMNISQL-32B. These baselines are used to examine whether OPTSQL, without task-specific fine-tuning, can achieve competitive performance compared with trained Text-to-SQL models.
- **Prompting-based methods.** This category contains training-free methods that adapt general-purpose LLMs to Text-to-SQL at inference time through prompting, schema serialization, demonstration retrieval, and reasoning instructions. We include **DIN-SQL** [32] and **DAIL-SQL** [8].
- **Agentic Text-to-SQL workflows.** We further compare with agent-based Text-to-SQL systems, including **MAC-SQL** [44] and **DEEPEYE** [15]. These methods decompose Text-to-SQL into multiple agentic steps, such as schema linking, SQL generation, execution-based refinement, and candidate selection. They provide strong baselines for evaluating whether the meta-cognitive controller and optimization loop in OPTSQL bring additional benefits beyond existing agentic workflows.

To ensure a fair comparison, for prompting-based methods and agentic workflows, we evaluate all systems under the same backbone LLMs. Specifically, we use three representative backbones: QWEN3-CODER-30B-A3B-INSTRUCT, DEEPSEEK-V4-PRO, and GPT-5.4.

Environment. We perform our experiments on a workstation with an 8-core 11th Gen Intel(R) Core(TM) i7-11700 @ 2.50GHz processor, 64GB of memory, and NVIDIA RTX A6000 with 48GB of VRAM running Ubuntu 22.04.1 LTS. For open-source LLMs, we deploy a local API server based on vLLM [11] which is a unified library for LLM serving and inference. For GPT-5.4, we query the respective official APIs with matching decoding settings. All experiments, including those for OPTSQL and every baseline, are repeated in three complete end-to-end runs, and we report the mean of each metric across the three runs. Each run regenerates all LLM outputs with fixed model versions and decoding settings. To reduce AEKB-induced dependence on a particular test order, we independently shuffle the evaluation instances for each run, use the same order across methods within that run, and reset AEKB to the same initial state before each run.

4.2 Performance Comparison

Overall Results. Table 2 reports the overall performance of OPTSQL and representative baselines on BIRD and EESQLBENCH. EX directly measures how often the generated SQL returns the correct result, while VES can be viewed as a runtime-aware accuracy score: incorrect SQL queries contribute zero, and correct SQL queries receive higher scores when they run faster than the canonical SQL. CR focuses on the semantically correct queries only, measuring whether their plan-level cost is close to or lower than the canonical SQL. AR@0.8 and AR@1 further measure the end-to-end fraction of test cases where the generated SQL is both correct and sufficiently efficient. Overall, OPTSQL achieves the strongest or highly competitive results across both correctness-oriented and efficiency-oriented metrics. On BIRD, OPTSQL achieves the best EX of 73.29%, the best VES of 83.02,

and the best AR@0.8 and AR@1 of 69.85% and 67.96%, respectively. On EESQLBENCH, OPTSQL achieves the best EX of 67.89%, the best VES of 107.16, the highest CR of 13.05, and the best AR@0.8 and AR@1 of 54.19% and 50.59%, respectively. These results show that OPTSQL improves not only SQL correctness but also efficiency.

Controlled Comparison under Same Backbones. We further compare methods under the same backbone LLMs to examine whether the improvements come from the workflow design rather than the underlying model. On BIRD, compared with the strongest agentic baseline DEEPEYE, OPTSQL improves EX by 0.79, 1.99, and 1.53 percentage points under QWEN3-CODER, DEEPSEEK-V4, and GPT-5.4, respectively. It also improves AR@1 by 0.66, 0.67, and 1.28 percentage points under the same three backbones. Compared with prompting-based methods such as DIN-SQL and DAIL-SQL, OPTSQL achieves substantially higher EX and AR@1, showing stronger end-to-end correctness and efficiency reachability. Moreover, compared with fine-tuning-based methods such as XiYAN-SQL and OMNI-SQL, OPTSQL achieves better overall performance without task-specific fine-tuning, improving the best fine-tuned baseline by 14.68 percentage points in EX and 11.31 percentage points in AR@1. On the more complex and efficiency-oriented EESQLBENCH, the advantage of OPTSQL becomes more pronounced. Under the same backbone LLMs, OPTSQL improves EX over DEEPEYE by 2.53, 1.12, and 3.39 percentage points, and improves AR@1 by 4.40, 1.98, and 0.58 percentage points under QWEN3-CODER, DEEPSEEK-V4, and GPT-5.4, respectively. The gains are larger on efficiency-oriented metrics, where OPTSQL improves VES by 18.05, 3.61, and 38.49 points over DEEPEYE under the same backbones. These results show that OPTSQL is robust across different backbone LLMs and significantly outperforms prompting-based, fine-tuning-based, and agentic baselines in both correctness and efficiency-oriented performance. Although some gains are modest, we view them as meaningful incremental progress on an extensively studied task, consistent with the small but measurable contributions of individual workflow components reported in recent Text-to-SQL systems [15, 31].

Statistical Significance. We conduct paired analysis between OPTSQL and the strongest baseline under the same backbone LLMs. For EX and AR@1, we use McNemar’s test over paired success outcomes; for VES and CR scores, we use the Wilcoxon signed-rank test over paired scores. The main comparisons are statistically significant ($p < 0.001$), and the 95% bootstrap confidence intervals are consistently positive, with a medium effect size of approximately $\delta = 0.52$.

Token Consumption. We compare the average token consumption per task on EESQLBENCH under QWEN3-CODER. OPTSQL consumes only 613K tokens per task on average, reducing token usage by 63.7% over MAC-SQL and 89.8% over DEEPEYE. This shows the advantage of adaptive workflow control: OPTSQL avoids applying expensive reasoning steps uniformly, and instead allocates evidence construction, reflection, and optimization only when they are needed.

4.3 Ablation Study

To understand the contribution of each core component in OPTSQL, we conduct an ablation study on both BIRD and EESQLBENCH using GPT-5.4 as the backbone LLM. We report EX to measure semantic correctness and AR@1 to measure the fraction of generated SQL queries that are both correct and no more expensive than the canonical SQL. We compare the full OPTSQL with four ablated variants:

- **w/o Adaptation:** replaces the controller’s state-dependent strategy selection with the same predefined workflow for all tasks. The choice and ordering of strategies are fixed rather than adapted to task context or intermediate feedback; SQL generation, local repair, and rewrite validation remain available within this workflow.

Table 2. **Overall performance comparison on BIRD and EESQLBENCH.** For both benchmarks, we report EX, VES, CR, AR@0.8, and AR@1. The best results are highlighted in bold.

Method	LLM	BIRD					EESQLBENCH				
		EX(%)	VES	CR	AR@0.8(%)	AR@1(%)	EX(%)	VES	CR	AR@0.8(%)	AR@1(%)
Fine-tuning-based methods											
XiYAN-SQL	XiYAN-32B	58.61	71.28	1.01	57.37	56.65	16.13	25.24	3.35	16.12	16.12
OMNI-SQL	OMNISQL-32B	49.41	57.03	1.00	47.45	46.48	15.05	20.87	1.44	14.52	14.52
Prompting-based methods											
DIN-SQL	QWEN3-CODER	57.63	64.29	1.86	54.41	52.80	18.29	47.63	5.81	13.50	12.53
	DEEPSEEK-V4	59.06	66.43	1.77	56.30	55.00	40.85	31.15	6.86	39.80	29.41
	GPT-5.4	42.05	46.42	1.42	39.81	38.62	49.39	67.44	10.92	38.16	36.81
DAIL-SQL	QWEN3-CODER	60.63	66.32	1.50	56.01	54.31	43.90	53.39	4.29	32.70	31.52
	DEEPSEEK-V4	39.99	65.64	2.25	38.73	37.81	54.88	47.78	7.62	38.50	37.12
	GPT-5.4	62.26	68.61	1.70	59.51	57.50	57.32	69.11	10.64	40.62	39.91
Agentic Text-to-SQL workflows											
MAC-SQL	QWEN3-CODER	65.71	72.21	1.83	61.40	59.12	57.32	51.68	8.04	39.10	37.51
	DEEPSEEK-V4	63.89	74.14	2.06	57.42	55.60	59.15	49.01	6.86	40.22	38.62
	GPT-5.4	66.56	75.47	1.87	59.60	57.12	60.37	49.92	8.49	40.82	36.20
DEEPEYE	QWEN3-CODER	72.50	81.52	1.69	69.60	67.30	59.76	62.26	7.37	46.70	46.19
	DEEPSEEK-V4	69.56	77.35	1.69	65.48	63.24	62.13	56.55	6.94	47.30	46.12
	GPT-5.4	68.45	74.94	1.70	63.41	61.30	64.50	68.67	3.96	46.30	43.31
OprSQL	QWEN3-CODER	73.29	82.42	1.70	69.85	67.96	62.29	80.31	13.05	51.42	50.59
	DEEPSEEK-V4	71.55	83.02	1.74	66.05	63.91	63.25	60.16	10.70	54.19	48.10
	GPT-5.4	69.98	76.32	1.72	64.85	62.58	67.89	107.16	6.76	51.62	43.89

Table 3. **Ablation study on BIRD and EESQLBENCH.** We use GPT-5.4 as the backbone LLM and report EX and AR@1.

Variant	BIRD		EESQLBENCH	
	EX(%)	AR@1(%)	EX(%)	AR@1(%)
Full	69.98	62.58	67.89	43.89
w/o Adaptation	68.45	61.82	65.24	40.34
w/o Reflection	67.32	58.15	58.80	36.89
w/o Optimization	69.98	57.25	67.89	25.05
w/o AEKB	69.98	59.60	67.89	38.65

- **w/o Reflection:** removes the controller’s explicit process-level reflection record r , while retaining execution-error repair within the SQL Builder and the Validator’s execution, consistency, and cost checks.
- **w/o Optimization:** returns the initial query selected by the SQL Builder at the end of the Generation Phase.
- **w/o AEKB:** disables retrieval of historical rewrite cases from the AEKB.

Table 3 shows that all core components contribute to the final performance of OptSQL. Removing workflow adaptation and process-level reflection both reduce EX and AR@1, indicating that dynamic strategy selection and intermediate self-reflection are important for handling diverse database environments and complex reasoning paths. In contrast, disabling the Optimization Loop or AEKB keeps EX unchanged but lowers AR@1, suggesting that these components mainly improve efficiency while preserving semantic correctness. The large AR@1 drop on EESQLBENCH, from 43.89% to 25.05% without optimization and to 38.65% without AEKB, confirms the benefit of plan-aware rewriting and reusable optimization experience for efficiency-oriented tasks.

4.4 Error Analysis

To better understand the limitations of OptSQL, we conduct a qualitative error analysis from both correctness and efficiency perspectives. Following a standard open-coding protocol, three annotators first independently propose fine-grained labels for recurring error patterns, and then iteratively compare, refine, and merge overlapping labels through discussion.

For **correctness analysis**, we randomly sample 100 cases judged as incorrect by EX and manually inspect the question, evidence, predicted SQL, and gold SQL. After resolving disagreements, we consolidate the errors into two high-level groups and 16 fine-grained labels: intent-violating hallucination (87%), where the predicted SQL changes the user intent, and potential gold error (13%), where the reference SQL is likely incomplete, ambiguous, or inconsistent with the question. The most representative fine-grained categories are join logic hallucination (21%) and DISTINCT error (15%), suggesting that join-path reasoning and duplicate semantics remain major sources of correctness failures.

For **inefficiency analysis**, we randomly sample 100 semantically correct cases with $CR_i < 1.0$ and compare the generated SQL, expert SQL, and their execution plans. Following the same coding protocol, we consolidate recurring inefficiency patterns into five high-level themes and 23 fine-grained labels: join logic (58%), subquery logic (5%), timing strategy (5%), access strategy (4%), and others (28%). The dominant fine-grained causes are redundant lookup joins (48%) and predicate pull-down opportunities (23%), indicating that many remaining inefficiencies come from conservative join construction and missed filter-placement rewrites. We hypothesize that the failures mainly stem from the limited capabilities of individual specialized agents, such as insufficient join-path reasoning and duplicate-semantics analysis. In future work, we plan to explore more specialized agents with stronger schema grounding, semantic validation, and SQL optimization capabilities. Due to space limitations, we release the detailed coding results, label definitions, and examples in our GitHub repository [28].

5 Discussion

Practical Impact. Following EESQLBENCH [21], we evaluate the correctness and execution efficiency of generated SQL, while recognizing that generation and optimization overhead also matters for practical deployment. These costs arise at different stages: SQL retained for subsequent use need not be regenerated or reoptimized before every execution. BayesQO [41] motivates investing additional offline optimization effort in recurring analytical queries, where even modest improvements in execution time can accumulate over repeated use. Similarly, GenRewrite [22] targets recurring workloads, including business-intelligence dashboards, reporting, and ETL, where the overhead of LLM-based rewriting can be amortized by reusing optimized SQL. Accordingly, OptSQL primarily targets database-backed business intelligence and analytics applications in which generated SQL is retained and repeatedly executed for dashboard refreshes, scheduled reports, and recurring data-processing tasks. In these workloads, semantically correct but inefficient SQL can create recurring bottlenecks, with unnecessary database work accumulating across executions and increasing latency and resource consumption. By combining correctness checks with execution-plan feedback and query rewriting, OptSQL aims to identify and reduce these inefficiencies under realistic schema and workload settings, addressing performance issues that correctness-only Text-to-SQL evaluation can overlook [21].

6 Threats to Validity

Internal Threats. The main internal threat lies in the use of EX as the primary correctness metric. Although EX is widely used in Text-to-SQL evaluation, it may not always faithfully reflect semantic correctness, since the gold SQL can be imperfect, or an incorrect predicted SQL may coincidentally produce the same execution result as the gold SQL. To assess this risk, we randomly inspected 100 evaluated cases and found that 95 cases were correctly judged by EX, suggesting that the impact of this issue is limited in our evaluation.

External Threats. External threats concern the potential data leakage of public benchmarks. Since some backbone LLMs may have been exposed to BIRD during pretraining, their performance

on BIRD may not fully reflect generalization to unseen tasks. To mitigate this risk, we additionally evaluate OPTSQL on EESQLBENCH, which was released after the expected knowledge cutoff of the selected models and thus reduces the likelihood of direct benchmark leakage. This additional benchmark helps assess the generality and efficiency-oriented capability of OPTSQL under a less leakage-prone setting.

Construct Validity. Expert-optimized reference queries may retain optimization opportunities, so their quality affects the interpretation of reference-relative efficiency scores and attainment thresholds. In particular, reaching or exceeding an expert reference does not establish that a generated query has minimum execution time or cost among all correct alternatives. Our cost-based metrics also capture only the selected aspect of execution efficiency: Total Scanned Rows does not account for every source of latency, so we interpret it alongside runtime-based VES rather than as an exact substitute for execution time.

7 Related Work

Fine-tuned and Prompting-based Text-to-SQL Methods. Fine-tuned methods adapt or train models with supervised fine-tuning, synthetic data, or reinforcement learning [5, 16, 18, 23, 24, 33, 34, 39, 42, 51]. However, these methods depend heavily on training data quality and coverage, and may generalize poorly to out-of-domain or complex real-world databases [15]. Prompting-based methods adapt LLMs without updating parameters by organizing schemas, sampled contents, demonstrations, decomposition hints, and retrieved evidence into effective prompts [8, 20, 25, 31, 32, 36, 39, 40, 46, 48]. Although these methods are training-free, their performance remains sensitive to prompt design, evidence quality, and backbone model capability.

Agentic Text-to-SQL Workflows. Recent work further extends prompting into agentic workflows, where LLMs inspect database context, call tools, execute SQL, observe feedback, and revise outputs. General agent frameworks such as REACT [50], SELF-REFINE [26], and REFLEXION [38] show the value of combining reasoning, acting, and iterative refinement. Following this paradigm, Text-to-SQL agents decompose SQL construction into modules such as schema exploration, candidate generation, execution-based repair, multi-path reasoning, semantic memory, and agentic view construction [2, 3, 6, 9, 29, 31, 44, 49]. Although these systems are more robust than single-pass prompting, they usually follow largely predefined workflows, use feedback mainly for post-generation repair or candidate selection, and pay limited attention to query efficiency.

Difference from OPTSQL. Existing prompting-based and agentic Text-to-SQL methods mainly focus on improving correctness through better grounding, decomposition, and post-hoc refinement. In contrast, OPTSQL is designed as an efficiency- and process-aware framework that goes beyond static or fixed workflows. It introduces a meta-cognitive controller that enables (i) environment-aware workflow adaptation, (ii) process-level self-reflection during generation, and (iii) execution- and plan-aware optimization guided by database feedback. As a result, OPTSQL treats Text-to-SQL as a dynamic interaction process where both reasoning strategy and optimization behavior are continuously adjusted according to the database environment and intermediate execution signals, rather than following a predefined pipeline or relying only on post-generation correction.

8 Conclusion

In this paper, we propose OPTSQL, a meta-cognitive multi-agent Text-to-SQL system that dynamically coordinates evidence construction, SQL generation, process-level reflection, and plan-aware optimization. Extensive experiments on BIRD and EESQLBENCH demonstrate that OPTSQL improves both correctness and query efficiency, highlighting the importance of meta-cognitive control for practical Text-to-SQL systems. We make our source code publicly available at <https://github.com/OptSQL/OptSQL/>.

Data Availability

The source code, evaluation scripts, case study, and error-analysis materials for OPTSQL are publicly available at <https://github.com/OptSQL/OptSQL/>.

References

- [1] Ion Androutsopoulos, Graeme D Ritchie, and Peter Thanisch. 1995. Natural language interfaces to databases—an introduction. *Natural language engineering* 1, 1 (1995), 29–81.
- [2] Asim Biswal, Chuan Lei, Xiao Qin, Aodong Li, Balakrishnan Narayanaswamy, and Tim Kraska. 2026. AgentSM: Semantic Memory for Agentic Text-to-SQL. *arXiv preprint arXiv:2601.15709* (2026).
- [3] Saumya Chaturvedi, Aman Chadha, and Laurent Bindschaedler. 2025. SQL-of-Thought: Multi-agentic Text-to-SQL with Guided Error Correction. *arXiv preprint arXiv:2509.00581* (2025).
- [4] Surajit Chaudhuri. 1998. An overview of query optimization in relational systems. In *Proceedings of the seventeenth ACM SIGACT-SIGMOD-SIGART symposium on Principles of database systems*. 34–43.
- [5] Xiaojun Chen, Tianle Wang, Tianhao Qiu, Jianbin Qin, and Min Yang. 2024. Open-SQL Framework: Enhancing Text-to-SQL on Open-source Large Language Models. *arXiv preprint arXiv:2405.06674* (2024).
- [6] Minghang Deng, Ashwin Ramachandran, Canwen Xu, Lanxiang Hu, Zhewei Yao, Anupam Datta, and Hao Zhang. 2025. ReFoRCE: A Text-to-SQL Agent with Self-Refinement, Consensus Enforcement, and Column Exploration. *arXiv preprint arXiv:2502.00675* (2025).
- [7] John H. Flavell. 1979. Metacognition and cognitive monitoring: A new area of cognitive-developmental inquiry. *American Psychologist* 34, 10 (1979), 906–911.
- [8] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2023. Text-to-sql empowered by large language models: A benchmark evaluation. *arXiv preprint arXiv:2308.15363* (2023).
- [9] Omid Reza Heidari, Siobhan Reid, and Yassine Yaakoubi. 2025. AGENTIQL: An Agent-Inspired Multi-Expert Framework for Text-to-SQL Generation. *arXiv preprint arXiv:2510.10661* (2025).
- [10] Gary G Hendrix, Earl D Sacerdoti, Daniel Sagalowicz, and Jonathan Slocum. 1978. Developing a natural language interface to complex data. *ACM Transactions on Database Systems (TODS)* 3, 2 (1978), 105–147.
- [11] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.
- [12] Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, et al. 2024. Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows. *arXiv preprint arXiv:2411.07763* (2024).
- [13] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2015. How good are query optimizers, really? *Proceedings of the VLDB Endowment* 9, 3 (2015), 204–215.
- [14] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. 9459–9474.
- [15] Boyan Li, Chong Chen, Zhujun Xue, Yinan Mei, and Yuyu Luo. 2026. DeepEye-SQL: A software-engineering-inspired text-to-sql framework. *Proceedings of the ACM on Management of Data* 4 (2026), 1–28.
- [16] Haoyang Li, Shang Wu, Xiaokang Zhang, Xinmei Huang, Jing Zhang, Fuxin Jiang, Shuai Wang, Tieying Zhang, Jianjun Chen, Rui Shi, et al. 2025. OmniSQL: Synthesizing High-quality Text-to-SQL Data at Scale. *arXiv preprint arXiv:2503.02240* (2025).
- [17] Haoyang Li, Jing Zhang, Cuiping Li, and Hong Chen. 2023. Resdsql: Decoupling schema linking and skeleton parsing for text-to-sql. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 13067–13075.
- [18] Haoyang Li, Jing Zhang, Hanbing Liu, Ju Fan, Xiaokang Zhang, Jun Zhu, Renjie Wei, Hongyan Pan, Cuiping Li, and Hong Chen. 2024. Codes: Towards building open-source language models for text-to-sql. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–28.
- [19] Jinyang Li, Binyuan Hui, Reynold Cheng, Bowen Qin, Chenhao Ma, Nan Huo, Fei Huang, Wenyu Du, Luo Si, and Yongbin Li. 2023. Graphix-t5: Mixing pre-trained transformers with graph-aware layers for text-to-sql parsing. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 37. 13076–13084.
- [20] Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2023. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems* 36 (2023), 42330–42357.
- [21] Li Lin, Yunfeng Shen, Lingfeng Bao, Rongxin Wu, and Yang Liu. 2026. Cracking Query Bottlenecks: Towards Efficiency-Oriented Text-to-SQL Generation. In *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*.

- [22] Jie Liu and Barzan Mozafari. 2025. GenRewrite: Query Rewriting via Large Language Models. *arXiv preprint arXiv:2403.09060* (2025). <https://arxiv.org/abs/2403.09060v3> Version 3, December 2025.
- [23] Yifu Liu, Yin Zhu, Yingqi Gao, Zhiling Luo, Xiaoxia Li, Xiaorong Shi, Yuntao Hong, Jinyang Gao, Yu Li, Bolin Ding, and Jingren Zhou. 2025. XiYan-SQL: A Novel Multi-Generator Framework for Text-to-SQL. *arXiv preprint arXiv:2507.04701* (2025).
- [24] Peixian Ma, Xialie Zhuang, Chengjin Xu, Xuhui Jiang, Ran Chen, and Jian Guo. 2025. SQL-R1: Training Natural Language to SQL Reasoning Model by Reinforcement Learning. *arXiv preprint arXiv:2504.08600* (2025).
- [25] Karime Maamari, Fadhil Abubaker, Daniel Jaroslawicz, and Amine Mhedhbi. 2024. The Death of Schema Linking? Text-to-SQL in the Age of Well-Reasoned Language Models. *arXiv preprint arXiv:2408.07702* (2024).
- [26] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-Refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems* 36 (2023), 46534–46594.
- [27] OptSQL Team. 2026. OptSQL Case Study. <https://github.com/OptSQL/OptSQL/blob/main/Case%20Study.md>. Accessed: 2026-06-30.
- [28] OptSQL Team. 2026. OptSQL Error Analysis. <https://github.com/OptSQL/OptSQL/blob/main/error%20analysis.md>. Accessed: 2026-06-30.
- [29] Minh Tam Pham, Trinh Pham, Tong Chen, Hongzhi Yin, Quoc Viet Hung Nguyen, and Thanh Tam Nguyen. 2026. AV-SQL: Decomposing Complex Text-to-SQL Queries with Agentic Views. *arXiv preprint arXiv:2604.07041* (2026).
- [30] Ana-Maria Popescu, Oren Etzioni, and Henry Kautz. 2003. Towards a theory of natural language interfaces to databases. In *Proceedings of the 8th international conference on Intelligent user interfaces*. 149–157.
- [31] Mohammadreza Pourreza, Hailong Li, Ruoxi Sun, Yeounoh Chung, Shayan Talaei, Gaurav Tarlok Kakkar, Yu Gan, Amin Saberi, Fatma Özcan, and Sercan O. Arik. 2024. CHASE-SQL: Multi-Path Reasoning and Preference Optimized Candidate Selection in Text-to-SQL. *arXiv preprint arXiv:2410.01943* (2024).
- [32] Mohammadreza Pourreza and Davood Rafiei. 2023. Din-sql: Decomposed in-context learning of text-to-sql with self-correction. *Advances in Neural Information Processing Systems* 36 (2023), 36339–36348.
- [33] Mohammadreza Pourreza and Davood Rafiei. 2024. DTS-SQL: Decomposed Text-to-SQL with Small Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP*.
- [34] Mohammadreza Pourreza, Shayan Talaei, Ruoxi Sun, Xingchen Wan, Hailong Li, Azalia Mirhoseini, Amin Saberi, Sercan O. Arik, et al. 2025. Reasoning-SQL: Reinforcement Learning with SQL Tailored Partial Rewards for Reasoning-Enhanced Text-to-SQL. *arXiv preprint arXiv:2503.23157* (2025).
- [35] Daking Rai, Bailin Wang, Yilun Zhou, and Ziyu Yao. 2023. Improving generalization in language model-based text-to-SQL semantic parsing: Two simple semantic boundary-based techniques. *arXiv preprint arXiv:2305.17378* (2023).
- [36] Ashwin Ramachandran and Sunita Sarawagi. 2024. Text-to-SQL Calibration: No Need to Ask – Just Rescale Model Probabilities. *arXiv preprint arXiv:2411.16742* (2024).
- [37] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. *arXiv preprint arXiv:2109.05093* (2021).
- [38] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, Vol. 36. 8634–8652.
- [39] Ruoxi Sun, Sercan Ö. Arik, Alex Muzio, Lesly Miculicich, Satya Gundabathula, Pengcheng Yin, Hanjun Dai, Hootan Nakhost, Rajarishi Sinha, Zifeng Wang, and Tomas Pfister. 2023. SQL-PaLM: Improved Large Language Model Adaptation for Text-to-SQL. *arXiv preprint arXiv:2306.00739* (2023).
- [40] Shayan Talaei, Mohammadreza Pourreza, Yu-Chen Chang, Azalia Mirhoseini, and Amin Saberi. 2024. Chess: Contextual harnessing for efficient sql synthesis. *arXiv preprint arXiv:2405.16755* (2024).
- [41] Jeffrey Tao, Natalie Maus, Haydn Jones, Yimeng Zeng, Jacob R. Gardner, and Ryan Marcus. 2025. Learned Offline Query Planning via Bayesian Optimization. *Proceedings of the ACM on Management of Data* 3, 3, Article 179 (2025), 29 pages. doi:10.1145/3725316
- [42] Dayton G. Thorpe, Andrew J. Duberstein, and Ian A. Kinsey. 2024. Dubo-SQL: Diverse Retrieval-Augmented Generation and Fine Tuning for Text-to-SQL. *arXiv preprint arXiv:2404.12560* (2024).
- [43] Francisco J. Varela, Evan Thompson, and Eleanor Rosch. 1991. *The Embodied Mind: Cognitive Science and Human Experience*. MIT Press.
- [44] Bing Wang, Changyu Ren, Jian Yang, Xinnian Liang, Jiaqi Bai, Linzheng Chai, Zhao Yan, Qian-Wen Zhang, Di Yin, Xing Sun, et al. 2023. Mac-sql: A multi-agent collaborative framework for text-to-sql. *arXiv preprint arXiv:2312.11242* (2023).
- [45] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171* (2022).

- [46] Yihan Wang, Peiyu Liu, and Xin Yang. 2025. LinkAlign: Scalable Schema Linking for Real-World Large-Scale Multi-Database Text-to-SQL. *arXiv preprint arXiv:2503.18596* (2025).
- [47] Zhaoguo Wang, Zhou Zhou, Yicun Yang, Haoran Ding, Gansen Hu, Ding Ding, Chuzhe Tang, Haibo Chen, and Jinyang Li. 2022. Wetune: Automatic discovery and verification of query rewrite rules. In *Proceedings of the 2022 International Conference on Management of Data*. 94–107.
- [48] Xiangjin Xie, Guangwei Xu, Lingyan Zhao, and Ruijie Guo. 2025. OpenSearch-SQL: Enhancing Text-to-SQL with Dynamic Few-shot and Consistency Alignment. *arXiv preprint arXiv:2502.14913* (2025).
- [49] Haolin Yang, Jipeng Zhang, Zhitao He, and Yi R. Fung. 2025. MARS-SQL: A multi-agent reinforcement learning framework for Text-to-SQL. *arXiv preprint arXiv:2511.01008* (2025).
- [50] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.
- [51] Zhewei Yao, Guoheng Sun, Lukasz Borchmann, Zheyu Shen, Minghang Deng, Bohan Zhai, Hao Zhang, Ang Li, and Yuxiong He. 2025. Arctic-Text2SQL-R1: Simple Rewards, Strong Reasoning in Text-to-SQL. *arXiv preprint arXiv:2505.20315* (2025).